

MongoDB Assignment

Connecting to MongoDB

Step 1: create node.js project

```
mkdir mongodb-project
```

```
cd mongodb-project
```

```
npm init -y
```

step 2: install required packages

```
npm install express mongoose dotenv
```

step 3: create MongoDB atlas cluster

go to:

Steps:

1. Create/Login Account
2. Create Free Cluster
3. Create Database User
4. Add IP Address → 0.0.0.0/0
5. Click **Connect**
6. Choose **Drivers**

7.Copy Connection String

Ex:

```
mongodb+srv://username:password@cluster0.mongodb.net/mydatabase?retryWrites=true&w=majority
```

step 4: create .env file

```
MONGO_URI=your_connection_string
```

```
PORT=3000
```

Step 5: server.js

```
const express = require("express");
```

```
const mongoose = require("mongoose");
```

```
const dotenv = require("dotenv");
```

```
dotenv.config();
```

```
const app = express();
```

```
// MongoDB Connection
mongoose
  .connect(process.env.MONGO_URI)
  .then(() => {
    console.log("MongoDB Connected
Successfully");
  })
  .catch((error) => {
    console.log("Connection Error:", error);
  });

app.get("/", (req, res) => {
  res.send("MongoDB Connection Working");
});

const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

```
});
```

Step 6: run project

```
node server.js
```

output:

MongoDB Connected Successfully

Server running on port 3000

Creating a model

Step 1: create models folder

```
mongodb-project/
```

```
|
```

```
├── models/
```

```
|   └── User.js
```

```
|
```

```
├── server.js
```

```
└── .env
```

Step 2: create models/user.js

```
const mongoose = require("mongoose");
```

```
// User Schema
```

```
const userSchema = new mongoose.Schema({
```

```
  name: {
```

```
    type: String,
```

```
    required: true,
```

```
  },
```

```
  email: {
```

```
    type: String,
```

```
    required: true,
```

```
    unique: true,
```

```
  },
```

```
  password: {
```

```
    type: String,
```

```
    required: true,
```

```
},  
});
```

```
// User Model
```

```
const User = mongoose.model("User",  
userSchema);
```

```
// Export Model
```

```
module.exports = User;
```

explanation of important lines

create schema

```
const userSchema = new mongoose.schema({})
```

used to define database fields

create model

```
const user=mongoose.model("user",  
userschema);
```

creates mongodb collection name is: users

export model

```
module.exports = user;
```

allows use of this model in other files

using model in server.js

```
const user=require("../models/user");
```

CRUD operations in mongodb using express & mongoose

Project Structure

project-folder/

|

├── models/

| └── User.js

|

├── server.js

├── .env

└── package.json

import required packages

server.js

```
const express = require("express");
```

```
const mongoose=require("mongoose");
```

```
const dotenv=require("dotenv");
```

```
dotenv.config();
```

```
const app= express();
```

```
app.use(express.json());
```

```
const user = require("../models/user");
```

connect MongoDB

mongoose

```
.connect(process.env.mongo_url)
```

```
.then(() => {
```

```
  Console.log("MongoDB connected successfully");
```

```
})
```

```
.catch((error) => {
```

```
  Console.log(error);});
```


Create and save documents

Post route

```
app.post("/users", async (req, res) => {  
  try {  
  
    const { name, email, password } = req.body;  
  
    const newUser = new User({  
      name,  
      email,  
      password,  
    });  
  
    const savedUser = await newUser.save();  
  
    res.status(201).json({  
      message: "User Created Successfully",  
      data: savedUser,  
    });  
  }  
});
```

```
    } catch (error) {  
  
        res.status(500).json({  
            error: error.message,  
        });  
  
    }  
});
```

Test POST API

URL

http://localhost:3000/users

Method

POST

Body (JSON)

```
{  
  "name": "Devika",  
  "email": "devika@gmail.com",
```

```
"password": "123456"  
}
```

Retrieve documents

Get route

```
app.get("/users", async (req, res) => {  
  try {  
  
    const users = await User.find();  
  
    res.status(200).json(users);  
  
  } catch (error) {  
  
    res.status(500).json({  
      error: error.message,  
    });  
  
  }  
}
```

```
});
```

Test GET API

URL

http://localhost:3000/users

Method

GET

3. Update Documents

PUT Route → /users/:id

```
app.put("/users/:id", async (req, res) => {  
  try {
```

```
    const updatedUser = await  
    User.findByIdAndUpdate(  
      req.params.id,  
      req.body,  
      { new: true }  
    );
```

```
    res.status(200).json({
```

```
        message: "User Updated Successfully",
        data: updatedUser,
    });

    } catch (error) {

        res.status(500).json({
            error: error.message,
        });

    }
});
```

Test PUT API

URL

http://localhost:3000/users/USER_ID

Method

PUT

Body (JSON)

```
{  
  "name": "Updated Name"  
}
```

4. Delete Documents

DELETE Route → /users/:id

```
app.delete("/users/:id", async (req, res) => {  
  try {  
  
    await User.findByIdAndDelete(req.params.id);  
  
    res.status(200).json({  
      message: "User Deleted Successfully",  
    });  
  
  } catch (error) {  
  
    res.status(500).json({  
      error: error.message,  
    });  
  
  }  
});
```

Test DELETE API

URL

http://localhost:3000/users/USER_ID

Method

DELETE

Server.js

```
const express = require("express");
const mongoose = require("mongoose");
const dotenv = require("dotenv");

dotenv.config();

const app = express();

app.use(express.json());
```

```
const User = require("../models/User");
```

```
// MongoDB Connection
```

```
mongoose
```

```
  .connect(process.env.MONGO_URI)
```

```
  .then(() => {
```

```
    console.log("MongoDB Connected  
Successfully");
```

```
  })
```

```
  .catch((error) => {
```

```
    console.log(error);
```

```
  });
```

```
// Home Route
```

```
app.get("/", (req, res) => {
```

```
  res.send("CRUD API Working");
```

```
});
```



```
// CREATE USER
```

```
app.post("/users", async (req, res) => {  
  try {
```

```
    const { name, email, password } = req.body;
```

```
    const newUser = new User({  
      name,  
      email,  
      password,  
    });
```

```
    const savedUser = await newUser.save();
```

```
    res.status(201).json({  
      message: "User Created Successfully",  
      data: savedUser,
```

```
});
```

```
  } catch (error) {
```

```
    res.status(500).json({  
      error: error.message,  
    });
```

```
  }
```

```
});
```

```
// GET USERS
```

```
app.get("/users", async (req, res) => {  
  try {
```

```
    const users = await User.find();
```

```
    res.status(200).json(users);
```

```
} catch (error) {  
  
  res.status(500).json({  
    error: error.message,  
  });  
  
}  
});  
  
// UPDATE USER  
app.put("/users/:id", async (req, res) => {  
  try {  
  
    const updatedUser = await  
    User.findByIdAndUpdate(  
      req.params.id,  
      req.body,
```

```
        { new: true }
    );

    res.status(200).json({
        message: "User Updated Successfully",
        data: updatedUser,
    });

} catch (error) {

    res.status(500).json({
        error: error.message,
    });

}

});

// DELETE USER
```

```
app.delete("/users/:id", async (req, res) => {  
  try {  
  
    await User.findByIdAndDelete(req.params.id);  
  
    res.status(200).json({  
      message: "User Deleted Successfully",  
    });  
  
  } catch (error) {  
  
    res.status(500).json({  
      error: error.message,  
    });  
  
  }  
});
```

```
// Start Server  
const PORT = process.env.PORT || 3000;  
  
app.listen(PORT, () => {  
  console.log(`Server running on port ${PORT}`);  
});
```

Run Project

```
node server.js
```

advanced MongoDB Assignment(authentication +
validation + pagination)

install required packages

```
npm install express mongoose dotenv bcryptjs  
jsonwebtoken express-validator
```

project-folder/

|

|— middleware/

| └─ auth.js

```
|  
|└─ models/  
|  └─ User.js  
|  
|└─ .env  
|└─ server.js
```

.env

MONGO_URI=your_mongodb_connection_string

JWT_SECRET=mysecretkey

PORT=3000

Models/user.js

```
const mongoose = require("mongoose");
```

```
const userSchema = new mongoose.Schema({  
  name: {  
    type: String,
```

```
    required: true,  
  },  
  
  email: {  
    type: String,  
    required: true,  
    unique: true,  
  },  
  
  password: {  
    type: String,  
    required: true,  
  },  
});  
  
module.exports = mongoose.model("User",  
userSchema);
```


authentication middleware

middleware/auth.js

```
const jwt = require("jsonwebtoken");
```

```
const auth = (req, res, next) => {
```

```
  const token = req.header("Authorization");
```

```
  if (!token) {
```

```
    return res.status(401).json({
```

```
      message: "Access Denied. No Token Provided",
```

```
    });
```

```
  }
```

```
  try {
```

```
    const verified = jwt.verify(token,  
    process.env.JWT_SECRET);
```

```
req.user = verified;
```

```
next();
```

```
} catch (error) {
```

```
res.status(400).json({  
  message: "Invalid Token",  
});
```

```
}
```

```
};
```

```
module.exports = auth;
```

```
server.js
```

```
const express = require("express");
```

```
const mongoose = require("mongoose");
```

```
const dotenv = require("dotenv");
```

```
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");

const { body, validationResult } = require("express-validator");

dotenv.config();

const app = express();

app.use(express.json());

const User = require("../models/User");
const auth = require("../middleware/auth");

// MongoDB Connection
mongoose
  .connect(process.env.MONGO_URI)
```

```
.then(() => {  
  console.log("MongoDB Connected  
Successfully");  
})  
.catch((error) => {  
  console.log(error);  
});
```

```
// Home Route
```

```
app.get("/", (req, res) => {  
  res.send("Advanced MongoDB API Working");  
});
```

```
// REGISTER USER
```

```
app.post(  
  "/register",
```

```
[
  body("name").notEmpty().withMessage("Name
is required"),

  body("email")
    .isEmail()
    .withMessage("Valid email is required"),

  body("password")
    .isLength({ min: 6 })
    .withMessage("Password must be at least 6
characters"),
],
```

```
async (req, res) => {
```

```
  const errors = validationResult(req);
```

```
  if (!errors.isEmpty()) {
```

```
return res.status(400).json({  
  errors: errors.array(),  
});  
}
```

```
try {
```

```
  const { name, email, password } = req.body;
```

```
  // Check Existing User
```

```
  const existingUser = await User.findOne({ email  
});
```

```
  if (existingUser) {
```

```
    return res.status(400).json({  
      message: "User already exists",  
    });
```

```
  }
```

```
// Hash Password
const salt = await bcrypt.genSalt(10);

const hashedPassword = await
bcrypt.hash(password, salt);

// Save User
const user = new User({
  name,
  email,
  password: hashedPassword,
});

await user.save();

res.status(201).json({
  message: "User Registered Successfully",
});
```

```
    } catch (error) {

        res.status(500).json({
            error: error.message,
        });

    }
}
);

// LOGIN USER
app.post("/login", async (req, res) => {

    try {

        const { email, password } = req.body;

        // Find User
```



```
const user = await User.findOne({ email });
```

```
if (!user) {  
  return res.status(400).json({  
    message: "Invalid Email or Password",  
  });  
}
```

```
// Compare Password
```

```
const validPassword = await bcrypt.compare(  
  password,  
  user.password  
);
```

```
if (!validPassword) {  
  return res.status(400).json({  
    message: "Invalid Email or Password",  
  });  
}
```

```
// Generate JWT
const token = jwt.sign(
  { id: user._id },
  process.env.JWT_SECRET,
  { expiresIn: "1h" }
);

res.status(200).json({
  message: "Login Successful",
  token,
});

} catch (error) {

  res.status(500).json({
    error: error.message,
  });
}
```

```
}  
});
```

```
// PROTECTED GET USERS ROUTE
```

```
app.get("/users", auth, async (req, res) => {
```

```
  try {
```

```
    // Query Parameters
```

```
    const { name, email, page = 1, limit = 5 } =  
    req.query;
```

```
    // Filter Object
```

```
    let filter = {};
```

```
    if (name) {
```

```
      filter.name = name;
```

```
    }
```

```
if (email) {  
  filter.email = email;  
}  
  
// Pagination  
const users = await User.find(filter)  
  .skip((page - 1) * limit)  
  .limit(Number(limit));  
  
res.status(200).json(users);  
  
} catch (error) {  
  
  res.status(500).json({  
    error: error.message,  
  });  
  
}
```

```
});
```

```
// UPDATE USER
```

```
app.put("/users/:id", auth, async (req, res) => {
```

```
  try {
```

```
    const updatedUser = await  
    User.findByIdAndUpdate(  
      req.params.id,  
      req.body,  
      { new: true }  
    );
```

```
    res.status(200).json({  
      message: "User Updated Successfully",  
      data: updatedUser,  
    });
```

```
    } catch (error) {  
  
        res.status(500).json({  
            error: error.message,  
        });  
  
    }  
});
```

// DELETE USER

```
app.delete("/users/:id", auth, async (req, res) => {  
  
    try {  
  
        await User.findByIdAndDelete(req.params.id);  
  
        res.status(200).json({
```

```
    message: "User Deleted Successfully",
  });

} catch (error) {

  res.status(500).json({
    error: error.message,
  });

}

});

// Start Server
const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Testing APIs in Postman

Post: <http://localhost:3000/register>

```
{  
  "name": "Devika",  
  "email": "devika@gmail.com",  
  "password": "123456"  
}
```

Login user

Post: <http://localhost:3000/login>

```
{  
  "email": "devika@gmail.com",  
  "password": "123456"  
}
```

Response

```
{  
  "message": "Login Successful",  
  "token": "your_jwt_token"
```



```
}
```

Access Protected Route

Get:

<http://localhost:3000/users?page=1&limit=5>

Aggregating Data using MongoDB

Use MongoDB aggregation to perform data analysis

Create aggregation route

Add route in server.js

```
app.get("/users/stats", async (req, res, next) => {
```

```
  try {
```

```
    const stats = await User.aggregate([
```

```
      // Group by null for total users
```

```
    {
```

```
    $group: {  
      _id: null,  
      totalUsers: { $sum: 1 },  
    },  
  },  
  
]);  
  
res.status(200).json({  
  message: "User Statistics",  
  data: stats,  
});  
  
} catch (error) {  
  
  next(error);  
  
}  
});
```

```
app.get("/users/domain-stats", async (req, res,  
next) => {
```

```
  try {
```

```
    const stats = await User.aggregate([
```

```
      {
```

```
        $project: {
```

```
          emailDomain: {
```

```
            $arrayElemAt: [
```

```
              { $split: ["$email", "@"] },
```

```
              1
```

```
            ]
```

```
          }
```

```
        }
```

```
      },
```

```
{
  $group: {
    _id: "$emailDomain",
    count: { $sum: 1 },
  }
}

]);

res.status(200).json(stats);

} catch (error) {

  next(error);

}

});
```

Output:

```
[  
  {  
    "_id": "gmail.com",  
    "count": 5  
  },  
  {  
    "_id": "yahoo.com",  
    "count": 2  
  }  
]
```

Handling errors in express.js

Why error handling is important

Error handling helps to:

Prevent application crashes

Show meaningful error messages

Improve security

Handle unexpected situations gracefully

Create centralized error middleware

project-folder/

|

|— middleware/

| — errorHandler.js

middleware/errorHandler.js

```
const errorHandler = (err, req, res, next) => {
```

```
  console.log(err.stack);
```

```
  res.status(500).json({
```

```
    success: false,
```

```
    message: err.message || "Internal Server Error",
```

```
  });
```

```
};
```

```
module.exports = errorHandler;
```

server.js

```
import middleware
const errorHandler =
require("./middleware/errorhandler");
app.use(errorHandler);
example route with error handling
app.get("/error", async (req, res, next) => {

  try {

    throw new Error("Something went wrong");

  } catch (error) {

    next(error);

  }

});
```

Error handling in node.js

Importance of error handling

Stable applications

Better user experience

Easier debugging

Controlled failures

Proper API responses

Types of error handling techniques

Try/catch: handles synchronous & async errors

Promise.catch(): handles promise errors

Middleware: centralized express error handling

Custom errors: create meaningful application errors

Logging: store error logs for debugging

Practical example:

Using try/catch

```
async function getUsers() {
```



```
try {  
  
    const users = await User.find();  
  
    console.log(users);  
  
} catch (error) {  
  
    console.log("Error:", error.message);  
  
}  
}
```

Using .catch()

```
User.find()  
    .then((users) => {  
        console.log(users);  
    })
```

```
.catch((error) => {  
  console.log(error.message);  
});
```

Testing and debugging

Testing helps:

Find bugs early

Improve code quality

Prevent future issues

Ensure API reliability

Make maintenance easier

Popular testing frameworks for node.js

Mocha: test runner

Chai: assertions

Jest: unit testing

Supertest: API testing

Install testing packages

```
npm install mocha chai supertest --save-dev
```

```
project-folder/
```

```
|
```

```
|— test/
```

```
|  └─ user.test.js
```

Example unit test

Test/user.test.js

```
const chai = require("chai");
```

```
const chaiHttp = require("chai-http");
```

```
const app = require("../server");
```

```
const expect = chai.expect;
```

```
chai.use(chaiHttp);
```

```
describe("User API Testing", () => {
```

```
  // Test Home Route
```

```
it("Should return home route", (done) => {
```

```
  chai
```

```
    .request(app)
```

```
    .get("/")
```

```
    .end((err, res) => {
```

```
      expect(res).to.have.status(200);
```

```
      done();
```

```
    });
```

```
  });
```

```
});
```

Middleware in express.js

What is middleware?

Middleware functions execute:

Before route handler

Between request and response cycle

Types of middleware

Express.json(): parse json data

Authentication middleware: verify jwt

Error middleware: handle errors

Logger middleware : log requests

Middleware example:

```
const logger = (req, res, next) => {  
  console.log(`${req.method} ${req.url}`);  
  next();  
};  
  
App.use(logger);
```

Middleware in express.js

What is middleware in express.js?

Middleware in express.js is a function that executes during the request-response cycle

It has access to:

Request object (req)

Response object(res)

Next middleware function(next)

Middleware can:

Execute code

Modify request/response objects

End requests

Pass control to the next middleware

Syntax of middleware

```
Const middleware = (req, res, next) = {
```

```
//code here
```

```
Next();
```

```
};
```

Role of middleware in request handling

Middleware helps to:

Log request information

Authenticate users

Validate user input

Handle errors

Parse JSON data

Protect routes

Types of middleware

Application middleware: runs for all routes

Route middleware: runs for specific routes

Error-middleware: handles applications error

Built-in middleware: provided by express

Third-party middleware: external packages

Examples of built in middleware:

`App.use(express.json());`

Used for parsing JSON request bodies

Practical assignment

Custom logging middleware

Create middleware/logger.js

```
const logger = (req, res, next) => {
```

```
const method = req.method;
```

```
const url = req.url;
```

```
const time = new Date().toLocaleString();
```

```
console.log(  
  `[${time}] ${method} ${url}`  
);
```

```
next();  
};
```

```
module.exports = logger;
```

```
server.js
```

```
const express = require("express");
```

```
const app = express();
```



```
const logger = require("../middleware/logger");
```

```
// Apply Middleware
```

```
app.use(logger);
```

```
app.get("/", (req, res) => {
```

```
  res.send("Middleware Working");
```

```
});
```

```
app.listen(3000, () => {
```

```
  console.log("Server running on port 3000");
```

```
});
```

Output: [5/28/2026, 10:30:20 AM] GET /

Authentication and security

Importance of authentication and security

Authentication and security are important because they:

- Protect user accounts

- Prevent unauthorized access

- Secure sensitive data

- Protect APIs from attacks

- Ensure user privacy

Common authentication strategies in node.js

- Session authentication: uses server sessions

- JWR authentication: uses tokens

- OAuth: login with google/github

- Passport.js: authentication middleware

JWT authentication

Used for:

Stateless authentication

Fast and secure

Easy API integration

Works well with frontend frameworks

Install required packages

```
npm install express mongoose bcryptjs  
jsonwebtoken dotenv
```

user model

models/user.js

```
const mongoose = require("mongoose");
```

```
const userSchema = new mongoose.Schema({
```

```
  name: {
```

```
    type: String,
```

```
    required: true,
```

```
  },
```

```
email: {  
  type: String,  
  required: true,  
  unique: true,  
},
```

```
password: {  
  type: String,  
  required: true,  
},
```

```
});
```

```
module.exports = mongoose.model("User",  
userSchema);
```

create .env

```
MONGO_URI=your_mongodb_connection_string
```

JWT_SECRET=mysecretkey

Registration and login routes

Server.js

```
const express = require("express");
```

```
const mongoose = require("mongoose");
```

```
const bcrypt = require("bcryptjs");
```

```
const jwt = require("jsonwebtoken");
```

```
const dotenv = require("dotenv");
```

```
dotenv.config();
```

```
const app = express();
```

```
app.use(express.json());
```

```
const User = require("../models/User");
```

```
// MongoDB Connection
```

```
mongoose
```

```
.connect(process.env.MONGO_URI)
```

```
.then(() => {
```

```
  console.log("MongoDB Connected");
```

```
})
```

```
.catch((error) => {
```

```
  console.log(error);
```

```
});
```

```
// Home Route
```

```
app.get("/", (req, res) => {
```

```
  res.send("Authentication API");
```

```
});
```

```
// REGISTER ROUTE
```

```
app.post("/register", async (req, res) => {
```

```
try {
```

```
    const { name, email, password } = req.body;
```

```
    // Check Existing User
```

```
    const existingUser = await User.findOne({ email  
});
```

```
    if (existingUser) {
```

```
        return res.status(400).json({  
            message: "User already exists",  
        });
```

```
    }
```

```
    // Hash Password
```

```
    const salt = await bcrypt.genSalt(10);
```

```
    const hashedPassword = await  
    bcrypt.hash(password, salt);
```

```
    // Create User
```

```
    const user = new User({  
      name,  
      email,  
      password: hashedPassword,  
    });
```

```
    await user.save();
```

```
    res.status(201).json({  
      message: "User Registered Successfully",  
    });
```

```
  } catch (error) {
```

```
    res.status(500).json({
```



```
        error: error.message,
    });

}

});

// LOGIN ROUTE
app.post("/login", async (req, res) => {

    try {

        const { email, password } = req.body;

        // Find User
        const user = await User.findOne({ email });

        if (!user) {
            return res.status(400).json({
```

```
    message: "Invalid Email or Password",
  });
}

// Compare Password
const validPassword = await bcrypt.compare(
  password,
  user.password
);

if (!validPassword) {
  return res.status(400).json({
    message: "Invalid Email or Password",
  });
}

// Generate JWT Token
const token = jwt.sign(
  { id: user._id },
```

```
    process.env.JWT_SECRET,  
    { expiresIn: "1h" }  
  );  
  
  res.status(200).json({  
    message: "Login Successful",  
    token,  
  });  
  
} catch (error) {  
  
  res.status(500).json({  
    error: error.message,  
  });  
  
}  
});
```

```
// Start Server
```

```
app.listen(3000, () => {
```

```
  console.log("Server running on port 3000");
```

```
});
```

Test APIs in postman

Register user : post

<http://localhost:3000/register>

output:

```
{
```

```
  "name": "Devika",
```

```
  "email": "devika@gmail.com",
```

```
  "password": "123456"
```

```
}
```

Login

<http://localhost:3000/login>

```
{  
  "email": "devika@gmail.com",  
  "password": "123456"  
}
```

Login response

```
{  
  "message": "Login Successful",  
  "token": "jwt_token_here"  
}
```